Cucumber für Performancetests

Manchmal ist es notwendig neue Funktionalitäten bezüglich ihrer Performance zu testen.
Das Hauptproblem hierbei ist es, genügend Testdaten zu generieren und die Verarbeitungszeit zu ermitteln.
In der anschliessenden Analyse sollen dann die zeitfressenden Prozesse ermittelt werden können.

Mit Cucumber Skripten + ein paar Python Skripten ist dies nun einfach möglich

- Testablauf
- Definition des Cucumber Skripts
- Definition der Testbettdatei
- Ausgabe Report
- Vorteile

Testablauf

1. Erstellen eines Cucumber Skriptes mit Tabellenbereichen `append row` bzw. `Example`-Tabellen eines `Scenario Outline`
2. Automatisches Erweitern des Cucumber Skriptes per Python Skript `pump_up.py`
3. Ausführen des "aufgeblasenen" Cucumber Skriptes
4. Auswerten der Ausführungszeiten über das Python Skript `cucumber_json.py`
   (Ausgewertet wird report/cucumber.json, die in jedem Cucumber re-Testverzeichnis generiert wird)

Definition des Cucumber Skripts

```
# ******************************************************************************
#  Name           : rahmen_perf.feature
#  Autor          : mibr
#  Verantwortlich : mibr
#  Kontrolle      : as
#  Funktion       : Testet Rahmenauftraege bezueglich der Performance.
#
# ******************************************************************************
#
Feature: RahmenauftaegePerformance
Background:
Given I set the fake date to "02.01.1995"
#  V E R K A U F

Scenario: VK Rahmenauftrag mit vielen offenen Auftraegen anlegen
# Kunde anlegen
Given I open an editor "KU1000" from table "(Customer):(Customer)" with command "NEW" for record ""
And I set fields
    | such    | KU1000             |
    | name    | Tony Tausend       |
    | ans     | Tony Tausend       |
    | nort    | Mille              |
    | plz     | 10000              |
And I save the current editor

Given I open an editor "RA1" from table "(Sales):(BlanketOrder)" with command "NEW" for record ""
And I set field "kunde" to id from editor "KU1000"
And I set field "such" to "RA1"
And I append rows
              | artex | mge  | verw  |
#LOOP_POS 1 to 1 |V1     | #id# | v#id# |
              |V1     | 1    | v1    |

And I save the current editor
```

```

Scenario Outline: VK RA mit vielen offenen AU Positionen
Given I open an editor "AU" from table "(Sales):(BlanketOrder)" with command "RELEASE" for record from editor "RA1"
And I set field "such" to "AUA<id>"
And I set field "mge" to "1" in row 1
And I save the current editor

Examples:
              | id   |
#LOOP 1 to 1  | #id# |
              | 1    |

Scenario: VK IS RAUOV zum KU1000 oeffnen
Given I open the infosystem "RAUOV"
And I set field "kkuli" to id from editor "KU1000"
And I press start
And I close the current editor
...
```

Besondere Beachtung finden die Zeilen

`#LOOP_POS 1 to 1 |V1     | #id# | v#id# |`

`#LOOP 1 to 1  | #id# |`

Diese werden später durch pump_up.py erweitert und wie in einer Schleife x-mal wiederholt. Dabei wird #id# durch den Iterationswert ersetzt.

`#LOOP_POS 1 to 4 |V1     | #id# | verw#id# |`

`wird zu`

`|V1 |  1 | vverw1 |`

`|V1 |  2 | vverw2 |`

`|V1 |  3 | vverw3 |`

`|V1 |  4 | vverw4 |`

Definition der Testbettdatei

[mm Testbett Doku]

```
NAME ref_ev_rahmen_perf_cu

VORGAENGER refbw1
NOREFSYS   QUIET
TESTGRUPPE performancetest
TESTGRUPPE cucumber

BESCHREIBUNG

  Verantwortlich : mibr

  Funktion : Performancetest Rahmenauftraege

  Beschreibung : Cucumber Tests fuer Rahmenauftraege mit vielen EK VK Vorgaengen.
                 Testet die Auswirkungen der Implementation der Rahmenauftragspsoitionsfelder fzahlabgerufen, fzahl

                 Testaufbau:
                 Das kurze rahmen_perf.feature Cucumber Skript wird in die Test-Root kopiert und mit dem Python Skr
                 um entsprechend viele Testfaelle erweitert (Zeilen mit LOOP , LOOP_POS werden vervielfacht).
```

```
                 Das pump_up.py Skript kann mit Faktoren aufgerufen werden, um so noch grössere Datenmengen zu erze
                 Ausgefuehrt wird das grosse erzeugte Cucumber Skript test/cucumber/ekvkservice/rahmen_perf_big.fea
                 Nach der Ausfuehrung erfolgt die Auswertung von report/cucumber.json, dass die einzelnen Ausfuehru
                 cucumber_json.py erzeugt eine uebersichtlichen Report.

IN
  #!include cucumber
  DIR test/cucumber/ekvkservice
  # unaufgepumptes Feature File in Testverzeichnis Root kopieren
  DATEI rahmen_perf.feature test/cucumber/ekvkservice

  # Umgebungsvariablen fuer python3 Aufrufe definieren
  ENVVAR python3 FEATUREPATH "$${REFSYSBASEDIR}ref_ev_rahmen_perf_cu/test/cucumber/ekvkservice"
  ENVVAR python3 PUMPUPPATH "$${HOMESRC}/python3/cucumber/performancetestsutils"
  ENVVAR python3 REFPATH "$${REFSYSBUILDDIR}/ref_ev_rahmen_perf_cu"

# Tools fuer Infosysteme
DATEI is/AUSWAHL
DATEI is/TABLEEREN
DATEI is/VERBOTEN
DATEI is/HOL.LISTE.EVPOS
DATEI is/EVT.FIELD.OUTCURR
DATEI is/EVT.FIELD.DEFAULT
DATEI is/EVT.FIELD.DECLARE

# Infosysteme  RAUOV, RAUOE
!include infosys/system.default
#!include infosys/ev/obligos.txt
#
#!include infosys/ev/rahmen.txt


INIT
  rm -rf rahmen_perf_big.feature ref_ev_rahmen_perf_cu.out ref_ev_rahmen_perf_cu.err.tmp ref_ev_rahmen_perf_cu.repo
  ${HOMESRC}/snaptool/initCucumberTest.sh

RUN
  # Feature Datei aufpumpen (Viele Testfaelle generieren)
  # Aufruf: python3 pump_up.py <Original Feature Datei> <aufgepumpte Feature Datei> [Faktor Example Zeilen LOOP] [F
  python3 $$PUMPUPPATH/pump_up.py $$REFPATH/rahmen_perf.feature $$FEATUREPATH/rahmen_perf_big.feature 100 100 >> re

  ${HOMESRC}/snaptool/runCucumberTest.sh

  # Ausgabe des Performance Ergebnisses
  python3 $$PUMPUPPATH/cucumber_json.py $$REFPATH/reports/cucumber.json scenario_durchlauf >> ref_ev_rahmen_perf_cu
  cat ref_ev_rahmen_perf_cu.report

OUT

DUMP
 ${HOMESRC}/std/tbin/taufraeum.sh -g
CMP

CLEAN

```

Beim Vergrössern des Skriptes können dem Skript zwei Faktoren per Argument übergeben werden werden:

`[Faktor Example Zeilen LOOP] Vervielfältigt die Wiederholung der LOOP Zeile -> Faktor = 2, LOOP 1 to 4 -> Wird 8 mal wiederholt`

`[Faktor Positionszeilen LOOP_POS] Vervielfältigt die Wiederholung der LOOP_POS`

Ausgabe Report

Mit dem Skript cucumber_json.py können die Zeiten aufsummiert ausgegeben werden.
Hierbei gibt es Zeiten für folgende Hierarchiestufen: Feature Datei > Szenario > Szenario Durchlauf (bei Szenario Outline) > Steps

```
Eingabedatei:   /m/dev-11/mibr/erp/x86_64_linux//ref_ev_rahmen_perf_cu/reports/cucumber.json

Feature File: 4min 26sek (266sek) - file:test/cucumber/ekvkservice/rahmen_perf_big.feature
Scenario: 1.93sek - VK Rahmenauftrag mit vielen offenen Auftraegen anlegen
Scenario Outline: 2min 33sek (153sek) - VK RA mit vielen offenen AU Positionen (100)
  Scenario Durchlauf: 1.47sek - VK RA mit vielen offenen AU Positionen
  Scenario Durchlauf: 2.02sek - VK RA mit vielen offenen AU Positionen
  ..
Scenario: 655ms - VK IS RAUOV zum KU1000 oeffnen
Scenario: 1.05sek - EK Rahmenauftrag mit vielen offenen Bestellungen anlegen
Scenario Outline: 1min 47sek (107sek) - EK RA mit vielen offenen BE Positionen (100)
  Scenario Durchlauf: 1.08sek - EK RA mit vielen offenen BE Positionen
  Scenario Durchlauf: 1.04sek - EK RA mit vielen offenen BE Positionen
  ..
Scenario: 2.35sek - EK IS RAUOE zum LI1000 oeffnen
```

Vorteile

- Das Ausgangs Cucumber Skript ist für sich selbst lauffähig und kann in Eclipse entwickelt werden (Die Beispielzeilen nach LOOP/LOOP_POS werden beim "aufpumpen" entfernt)
- Der Test ist über Faktoren skalierbar:
  Es können hunderte oder tausende Wiederholungen angelegt werden.
  Somit kann ermittelt werden, ob sich die Ausführungszeiten linear oder exponentiell verhalten
- Über die Faktoren können die Anzahl der Vorgänge oder aber die Anzahl der Tabellenpositionen separat gesteuert werden
- Zum Entwickeln kann der Test mit niedrigen Faktoren ausgeführt werden (Zeitgewinn bei der Entwicklung)
- Die Erstellung der Testvorgänge ist sehr flexibel, z.B. können unterschiedliche Vorgangsketten angelegt werden.
- Die Auswertung der Zeiten ist in unterschiedlichen Tiefen möglich und gut lesbar

Exemplarisch hierfür der Automatiktest: ref_ev_rahmen_perf_cu